

T.D 3 : Ensembles

Le type abstrait Ensemble correspond à l'idée d'un ensemble auquel on peut ajouter ou enlever des éléments. Chaque élément ne peut y appartenir qu'une fois. On peut déterminer si un élément appartient ou non à un ensemble.

Type abstrait Ensemble(T)

Opérations

- ensemble-vide : $\rightarrow$ Ensemble(T).
- est-vide : Ensemble(T) $\rightarrow$ Booléen
- ajouter : T x Ensemble(T) $\rightarrow$ Ensemble(T)
- supprimer : Ensemble(T) $\rightarrow$ Ensemble(T)
- appartient : T x Ensemble(E) $\rightarrow$ Booléen

Axiomes

- est-vide(ensemble-vide()) = Vrai
- est-vide(ajouter(x, e)) = Faux
- appartient(x, ensemble-vide()) = Faux
- appartient(x, ajouter(y, e)) = x == y ou appartient(x, e)
- ajouter(x, ajouter(x, e)) = ajouter(x, e)
- supprimer(x, ajouter(x, e)) = e si
- si x != y :
- ajouter(x, ajouter(y, e)) = ajouter(y, ajouter(x, e))
- si x != y :
- supprimer(x, ajouter(y, e)) = ajouter(y, supprimer(x, e))

Exercice 1

1. Implémenter une classe **Ensemble** qui ne s'applique qu'aux entiers positifs inférieur ou égal à un entier n donné. On utilisera une liste Python de booléens pour cette implémentation, on ne redimensionnera pas cette liste. **Ensemble(n)** doit renvoyer un ensemble vide permettant de travailler sur les entiers de 0 à n (inclus).
2. Ajouter à la classe **Ensemble** une méthode permettant d'ajouter à l'ensemble tous les éléments inclus dans une liste Python donnée.
3. Ajouter à la classe **Ensemble** une méthode **somme** renvoyant la somme de tous les éléments présent dans l'ensemble.
4. Modifier la classe de façon à pouvoir déterminer le nombre d'élément dans un ensemble en temps constant.

Exercice 2

- Dans cet exercice, on privilégiera dès que c'est possible les opérateurs bit à bit :
- **décalage à gauche de n bits** : `a << n` ; `3 << 1` renvoie 6
 - **décalage à droite de n bits** : `a >> n` ; `16 >> 2` renvoie 4
 - **et bit à bit** : `a & b` ; `13 & 11` renvoie 9
 - **ou bit à bit** : `a | b` ; `13 | 11` renvoie 15
 - **xor bit à bit** : `a ^ b` ; `13 ^ 11` renvoie 15
1. Implémenter une classe **Ensemble** qui ne s'applique qu'aux entiers positifs sans restriction sur la taille de ceux-ci. On utilisera la représentation binaire d'un **int** pour cette implémentation.

2. Opération ensembliste :

- a. Surcharger l'opérateur | pour qu'il renvoie l'union de deux ensembles.
- b. Implémenter l'intersection de deux ensembles via la surcharge d'un opérateur pertinent.
- c. Surcharger l'opérateur - pour qu'il renvoie la différence de deux ensembles.
- d. La différence symétrique entre deux ensembles E1 et E2, qui est notée E1ΔE2, représente l'ensemble des éléments qui appartiennent soit à E1, soit à E2 mais pas à E1 et E2 en même temps. Implémenter une méthode donnant la différence symétrique entre deux ensembles. Proposer un opérateur à surcharger pour cela.

3. Inclusions.

- a. Surcharger l'opérateur <= pour implémenter l'inclusion d'un ensemble dans un autre.
- b. Surcharger l'opérateur == pour implémenter l'égalité de deux ensembles.
- c. Implémenter une méthode déterminant si deux ensembles sont disjoints.
4. a. Décrire les bits de la représentation non signé de n-1 à partir de celle de n.
- b. Proposer une méthode efficace pour déterminer si un entier est une puissance de deux.
- c. Qu'est-ce que la fonction **mystere** permet de déterminer pour un entier n ?

```
0 def mystere(n):
1     cpt = 0
2     while n:
3         n &= n-1
4         cpt += 1
5     return cpt
6
```

- d. Permettre de déterminer le nombre d'éléments dans l'ensemble via **len**.

Exercice 3

1. Proposer une implémentation des ensembles à partir des dictionnaires Python.
2. Il existe déjà un type **set** en Python. Utiliser un **set** Python pour écrire une fonction déterminant le nombre de caractère différent dans une chaîne de caractère.